

Unsupervised Learning, Recommenders, RL

Course 3 — detailed study notes (Weeks 1–3)

Clustering · anomaly detection · recommenders · PCA · reinforcement learning

1. Week 1a — Clustering (K-means)

1.1 The algorithm

Unsupervised: only $\mathbf{x}^{(i)}$, no labels. Group data into K clusters. Two steps repeat until nothing changes:

1. **Assign:** each point $\mathbf{x}^{(i)}$ to its nearest centroid, $c^{(i)} = \arg \min_k \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_k\|^2$.
2. **Move:** each centroid $\boldsymbol{\mu}_k$ to the mean of the points assigned to it.

Initialize the K centroids by randomly picking K training examples.

1.2 Optimization objective (distortion)

K-means is minimizing one cost — the average squared distance to the assigned centroid:

$$J = \frac{1}{m} \sum_{i=1}^m \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_{c^{(i)}}\|^2.$$

The assign step minimizes J over the $c^{(i)}$; the move step minimizes J over the $\boldsymbol{\mu}_k$. J can never increase — so it always converges.

K-means can land in a bad local optimum from an unlucky start. **Fix:** run it 50–1000 times with different random inits and keep the run with the lowest J .

1.3 Choosing K

- **Elbow method:** plot J vs. K ; pick the “elbow” where the drop flattens. Often ambiguous.
- **Better:** choose K by the downstream purpose (e.g. how many T-shirt sizes S/M/L/XL you actually want to make).

2. Week 1b — Anomaly Detection

2.1 Gaussian (normal) distribution

$$p(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi} \sigma} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right), \quad \mu = \frac{1}{m} \sum_i x^{(i)}, \quad \sigma^2 = \frac{1}{m} \sum_i (x^{(i)} - \mu)^2.$$

2.2 The algorithm

Assume features are independent; multiply their Gaussians:

$$p(\mathbf{x}) = \prod_{j=1}^n p(x_j; \mu_j, \sigma_j^2) = \prod_{j=1}^n \frac{1}{\sqrt{2\pi} \sigma_j} \exp\left(-\frac{(x_j - \mu_j)^2}{2\sigma_j^2}\right).$$

Flag $\mathbf{x}$ as an anomaly if $p(\mathbf{x}) < \varepsilon$.

2.3 Developing & evaluating

Use a labeled CV/test set (a few known anomalies) even though training is unsupervised:

- Fit μ_j, σ_j^2 on the (mostly normal) **training** set.
- Tune ε on the **CV** set using F_1 (classes are very skewed, so accuracy is useless).

Anomaly detection vs. supervised learning

Anomaly detection

very few positive examples
many “types” of anomaly, future ones unseen
fraud, manufacturing defects, monitoring

Supervised learning

many positive examples
future positives look like past ones
spam, weather, known-class problems

Choosing features

- Make each feature more Gaussian: transform with $\log(x + c)$, $\sqrt{x}$, $x^{1/3}$, etc.
- If an anomaly slips through with high p , **engineer a new feature** that separates it (e.g. a ratio like CPU load / network traffic).

3. Week 2a — Collaborative Filtering

3.1 Setup

Users rate items; most entries missing. $r(i, j) = 1$ if user j rated item i ; $y^{(i,j)}$ = that rating. Learn a feature vector $\mathbf{x}^{(i)}$ per item and a parameter vector $\mathbf{w}^{(j)}, b^{(j)}$ per user, so the prediction is

$$\hat{y}^{(i,j)} = \mathbf{w}^{(j)} \cdot \mathbf{x}^{(i)} + b^{(j)}.$$

3.2 Cost — learn users *and* items together

The trick of collaborative filtering: you don’t know the item features either, so learn both at once:

$$J = \frac{1}{2} \sum_{(i,j):r(i,j)=1} (\mathbf{w}^{(j)} \cdot \mathbf{x}^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{j,k} (w_k^{(j)})^2 + \frac{\lambda}{2} \sum_{i,k} (x_k^{(i)})^2.$$

Gradient descent updates $\mathbf{w}^{(j)}, b^{(j)}$ and $\mathbf{x}^{(i)}$. Each user’s ratings help pin down item features, which in turn help predict for other users — hence “collaborative.”

3.3 Binary labels

For favs/likes/clicks, replace the squared error with logistic loss and predict $g(\mathbf{w}^{(j)} \cdot \mathbf{x}^{(i)} + b^{(j)})$ — same idea as going from linear to logistic regression.

3.4 Mean normalization

Subtract each item’s mean rating μ_i before training; add it back at prediction time. This makes sensible predictions for brand-new users (who otherwise get all zeros) — they get the average rating.

3.5 Finding related items

Items i and k are similar when $\|\mathbf{x}^{(i)} - \mathbf{x}^{(k)}\|^2$ is small — the learned feature vectors do the work.

Cold start: collaborative filtering struggles with brand-new items/users (no ratings yet) and can't use side information (user demographics, item genre). Content-based filtering addresses this.

4. Week 2b — Content-Based Filtering

4.1 Idea

Match *user features* $\mathbf{x}_u^{(j)}$ (age, genres liked) to *item features* $\mathbf{x}_m^{(i)}$ (genre, year). Instead of a dot product of raw features, pass each through a neural network to get equal-length vectors:

$$\mathbf{v}_u^{(j)} = \text{NN}_u(\mathbf{x}_u^{(j)}), \quad \mathbf{v}_m^{(i)} = \text{NN}_m(\mathbf{x}_m^{(i)}), \quad \hat{y}^{(i,j)} = \mathbf{v}_u^{(j)} \cdot \mathbf{v}_m^{(i)}.$$

The two networks (user tower + item tower) are trained jointly on the rating data. $\mathbf{v}_u$ and $\mathbf{v}_m$ must have the same dimension so the dot product works.

4.2 Large catalogues — retrieval + ranking

- **Retrieval:** quickly gather a few hundred plausible candidates (precompute $\mathbf{v}_m^{(i)}$ for all items offline; approximate nearest-neighbor search).
- **Ranking:** run the full model only on that shortlist to score and order them. Keeps it fast at scale.

Ethics: recommenders optimize what you tell them to. Maximizing watch-time or engagement can amplify harmful/addictive content; be deliberate about the objective and about transparency.

5. Week 2c — PCA (optional)

Reduce many features to a few (2-3) for **visualization**.

1. Mean-normalize (and feature-scale) the data.
2. Find the **principal components**: new axes (orthogonal directions) that capture the most variance.
3. Project each point onto the top k components to get its reduced coordinates.

The first PC is the direction of maximum variance; each next PC is orthogonal to the previous and captures the most remaining variance. “Explained variance ratio” tells you how much information each component retains.

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
X_2d = pca.fit_transform(X)
print(pca.explained_variance_ratio_)
```

PCA is for visualization/compression, *not* a fix for overfitting — use regularization for that.

6. Week 3 — Reinforcement Learning

6.1 Framework

An **agent** in **state** s takes **action** a , receives **reward** $R(s)$, and moves to a new state s' . No labeled “right answer” — the reward signal tells it how well it's doing. Goal: learn a **policy** $\pi(s) = a$ that maximizes long-term reward. (Mars rover / lunar lander examples.)

6.2 The Return (discounted reward)

Future rewards are worth less than immediate ones. With discount factor $\gamma \in (0, 1)$:

$$\text{Return} = R_1 + \gamma R_2 + \gamma^2 R_3 + \gamma^3 R_4 + \dots$$

γ near 1 = patient (values the future); smaller γ = greedy for near-term reward. Discounting also keeps the sum finite.

6.3 Policy

A policy π maps each state to an action, $\pi(s) = a$. RL's job is to find the π that maximizes the expected return from every state.

6.4 State-action value function $Q(s, a)$

$Q(s, a)$ = return if you start in s , take action a once, then act optimally after.

Once you have Q , the best action in state s is $\arg \max_a Q(s, a)$, and the best possible return from s is $\max_a Q(s, a)$.

6.5 Bellman equation

Q obeys a recursive relationship: immediate reward now, plus discounted best value of where you land:

$$Q(s, a) = R(s) + \gamma \max_{a'} Q(s', a').$$

This is the core equation of RL — it links each state's value to its successor's.

Stochastic environments (optional)

If actions don't always succeed (the rover may slip), maximize *expected* return; the Bellman equation gains an expectation: $Q(s, a) = R(s) + \gamma \mathbb{E}[\max_{a'} Q(s', a')]$.

6.6 Continuous states — Deep Q-Network (DQN)

When states are continuous (lunar lander: position, velocity, angle), tabulating Q is impossible — approximate it with a neural network.

1. Input the network (s, a) ; it outputs $Q(s, a)$. (Better: input s , output Q for *all* actions at once.)
2. Generate training tuples $(s, a, R(s), s')$ by interacting with the environment.
3. Target $y = R(s) + \gamma \max_{a'} Q(s', a')$ (Bellman). Train the net so $Q(s, a) \approx y$ via MSE.
4. Repeat — Q improves, so the policy improves.

ϵ -greedy policy

While learning, balance **exploration vs. exploitation**: with probability $1 - \epsilon$ pick the greedy action $\arg \max_a Q(s, a)$; with probability ϵ pick a random action. Often start ϵ high and decay it as Q becomes trustworthy.

Refinements (optional)

- **Better architecture:** one network outputting all $Q(s, a_k)$ together — one forward pass gives every action's value.
- **Mini-batch:** train on random subsets of stored experience each step (faster, more stable).
- **Soft updates:** nudge the target network's weights slowly toward the new ones to avoid oscillation.

State of RL: powerful in simulation and games, but harder on real robots (sample-hungry, sensitive). Fewer commercial applications than supervised learning today — but an active research frontier.

Three ideas of Course 3: (1) *unsupervised* methods find structure without labels — K-means minimizes distortion, anomaly detection thresholds a Gaussian $p(\mathbf{x})$; (2) *recommenders* learn user/item vectors whose dot product predicts ratings (collaborative when features are latent, content-based when you have side info); (3) *RL* learns a policy by maximizing discounted return, with $Q(s, a)$ and the Bellman equation $Q = R + \gamma \max_{a'} Q'$ at its heart.